SmartKDS & Menu

Documentação de Arquitetura de Software · arc42 · C4 Model · **Atualização da Entrega 3**

Arthur Sales Major · Vinicius Ogasawara · Luiz Eduardo Assi · Rodrigo Petrelli Glir

Novembro de 2026

Este documento complementa a documentação arc42 das entregas anteriores com o que foi implementado e publicado na Entrega 3: links, Software Architecture Canvas e C4 atualizados (contexto, contêineres e componentes), decisões novas e os prints exigidos. Para gerar o PDF, abra este arquivo no navegador e use *Imprimir → Salvar como PDF*.

1. Links da entrega

> Endereços da publicação na Azure (Azure for Students). Só o API Gateway e os microfrontends são públicos; BFFs e microsserviços ficam no ingresso interno do Container Apps. Falta apenas o link do vídeo.

Item	Link
GitHub — repositório público (monorepo: documentação, compose, infra e todos os projetos)	`https://github.com/Ogasawara23/ArquiteturaNuvem`
GitHub — Microfrontend	`https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/frontend`
GitHub — BFF	`https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/bff`
GitHub — Microsserviço Mongo (catálogo)	`https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/services/catalogo-service`
GitHub — Microsserviço SQL (pedido)	`https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/services/pedido-service`
GitHub — Azure Function	`https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/functions/status-operacional`
GitHub — API Gateway e demais serviços	`https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/gateway/api-gateway`, `https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/services/producao-service`, `https://github.com/Ogasawara23/ArquiteturaNuvem/tree/main/services/estabelecimento-service`
Docker Hub	`https://hub.docker.com/u/viniogasawara` — 15 imagens `smartkds-*`: 4 microsserviços, `api-gateway` e `status-operacional-function` (tag `v2`), 3 BFFs (tag `v3`) e 6 microfrontends (tag `fe-v1`)
API Gateway	`https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io`
Endpoint agregado	`https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/admin/aggregated-data`
Swagger — BFFs	`https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/admin/docs` · `/bff/mesa/docs` · `/bff/kds/docs`
Swagger — Microsserviços	`https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/docs/catalogo/swagger-ui.html` · `/docs/pedido/swagger-ui.html` · `/docs/producao/swagger-ui.html` · `/docs/estabelecimento/swagger-ui.html`
Swagger — Eventos (AsyncAPI 3.0)	`https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/admin/eventos`
Azure Function	`https://smartkds-status-5560.azurewebsites.net/api/status-operacional?tenantId=demo`

Frontend — operação (gerente, cozinha, atendente)	`https://shell-operacao.livelyforest-4b10050b.chilecentral.azurecontainerapps.io`
Frontend — mesa do cliente (QR Code)	`https://shell-mesa.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/m/demo-mesa-1`
Serviços gerenciados	Azure SQL Database `sql-smartkds-8759/smartkds_pedido` (Basic) · MongoDB Atlas M0 (AWS São Paulo) · PostgreSQL no Neon (AWS São Paulo) · LavinMQ na CloudAMQP (AWS São Paulo)
Vídeo no YouTube	`https://youtu.be/<id>`

1.1 Requisitos da Entrega 3 e onde estão atendidos

Requisito	Implementação
Microfrontend (React) que consome apenas o BFF	2 shells + 4 remotes (Module Federation). Os remotes só chamam `/bff/**` pelo gateway; testes de arquitetura proíbem `fetch` fora de `src/api` e import entre remotes.
BFF Node.js: agregação, proxy CRUD, microsserviços e Function	`bff-admin` (agregação + proxy CRUD), `bff-mesa` e `bff-kds` (composição + SSE). Node 22 + Fastify.
`GET /aggregated-data` (MS1 + MS2 + Function em um response)	`bff-admin`: chamadas em paralelo a `catalogo-service`, `pedido-service` e `status-operacional` (+ indicadores do `producao-service`), com situação e latência de cada fonte.
Microsserviço 1 — MongoDB Atlas, CRUD, Swagger	`catalogo-service`: POST/GET/GET por id/PUT/PATCH/DELETE de itens + versões imutáveis do cardápio.
Microsserviço 2 — Azure SQL, CRUD, Swagger	`pedido-service`: POST abrir comanda e adicionar item, GET listar/obter, PATCH alterar item, DELETE cancelar item/comanda, POST confirmar/fechar. Azure SQL com RLS por `SESSION_CONTEXT`.
Azure Function (HTTP Trigger)	`status-operacional`: faixa horária, pico e multiplicador do tempo de preparo.
API Gateway (roteamento, segurança, entrada única)	Spring Cloud Gateway: roteamento por prefixo, JWT do Keycloak, tenant sempre sobrescrito, CORS, rate limit, correlação.
Event-Driven Architecture	Broker AMQP 0-9-1 (LavinMQ na CloudAMQP, compatível com RabbitMQ) com Transactional Outbox, consumo idempotente (inbox), DLQ e saga coreografada; contrato AsyncAPI.
Clean Architecture + Vertical Slice	Domain = `shared/dominio` (Java puro); Application = `Handler` / `Query` da fatia; Infrastructure = `shared/persistencia|outbox|inbox|integracao`; API = `Endpoint` / `Listener`. Fatias em `features/<fatia>`.
Testes unitários e de arquitetura	JUnit + ArchUnit (13 a 18 regras por serviço), integração com PostgreSQL real e SQL Server (Testcontainers); Vitest nos BFFs, Function e frontends com testes de arquitetura por imports.
Docker Hub	Dockerfile por serviço/BFF/gateway/function; `scripts/publicar-imagens.sh` e workflow `cd.yml`.
Repositórios públicos e README	Repositório público único (monorepo) com uma pasta por projeto — Microfrontend, BFF, MS Mongo, MS SQL, Function, gateway — cada uma com README (arquitetura, tecnologias, execução e alunos), Dockerfile e job de CI próprios (ADR-17).

2. Software Architecture Canvas (atualizado)

Bloco	Conteúdo
Business Case	Reduzir o tempo entre o pedido e a entrega do prato, eliminando a comanda em papel: o cliente pede pelo celular, a cozinha recebe o ticket na hora e o gerente enxerga o gargalo em tempo real.
Business Context	Restaurantes de pequeno e médio porte (SaaS multi-tenant), mesas com QR Code, estações de produção (grill, fritura, bar, sobremesa) e equipe de salão e cozinha.
Functional Overview	Cardápio digital versionado, comanda eletrônica pelo QR Code, KDS por estação com SLA e alerta, CRUD completo de comandas, fechamento, painel agregado (`/aggregated-data`) e auditoria.
Quality Goals	1. Latência na cozinha · 2. Resiliência e degradação parcial · 3. Isolamento multi-tenant · 4. Evolutibilidade e deploy independente · 5. Observabilidade · 6. Economicidade
Organisational Constraints	Equipe de 4 alunos · um semestre · créditos estudantis (Azure for Students e camadas gratuitas) · LGPD
Technical Constraints	React 18 + Vite + Module Federation · BFFs Node.js 22 (Fastify) · Microsserviços Java 21 / Spring Boot 3.5 em Vertical Slice · Spring Cloud Gateway · **MongoDB Atlas** (catálogo) · **Azure SQL Database** (pedido) · PostgreSQL no Neon (produção, estabelecimento) · LavinMQ na CloudAMQP (AMQP, compatível com RabbitMQ) · **Azure Functions** · Keycloak · Docker Hub · Azure Container Apps (gateway, BFFs, serviços e microfrontends) · política do Azure for Students restrita a 5 regiões
Architecture Overview	Dois shells carregam remotes independentes; todo tráfego entra pelo API Gateway, que roteia para o BFF da experiência. O `bff-admin` agrega MS1, MS2 e a Azure Function. Cada microsserviço tem banco próprio; mudanças de estado são publicadas no broker AMQP (LavinMQ) via Transactional Outbox e consumidas de forma idempotente.
Architecture Hypotheses	Assíncrono entre pedido e produção absorve picos · BFF por experiência evita payloads inúteis · Vertical Slice reduz o custo de novas funcionalidades · MongoDB acomoda modificadores heterogêneos · Azure SQL garante transação conjunta agregado + outbox.
Technical Challenges & Risks	Consistência eventual · operação de 9 contêineres e 4 tipos de armazenamento · versionamento de contratos · SSE com reconexão · limites das camadas gratuitas (Azure SQL serverless pausa, CloudAMQP com poucas conexões).

3. C4 Nível 1 — Contexto (atualizado)

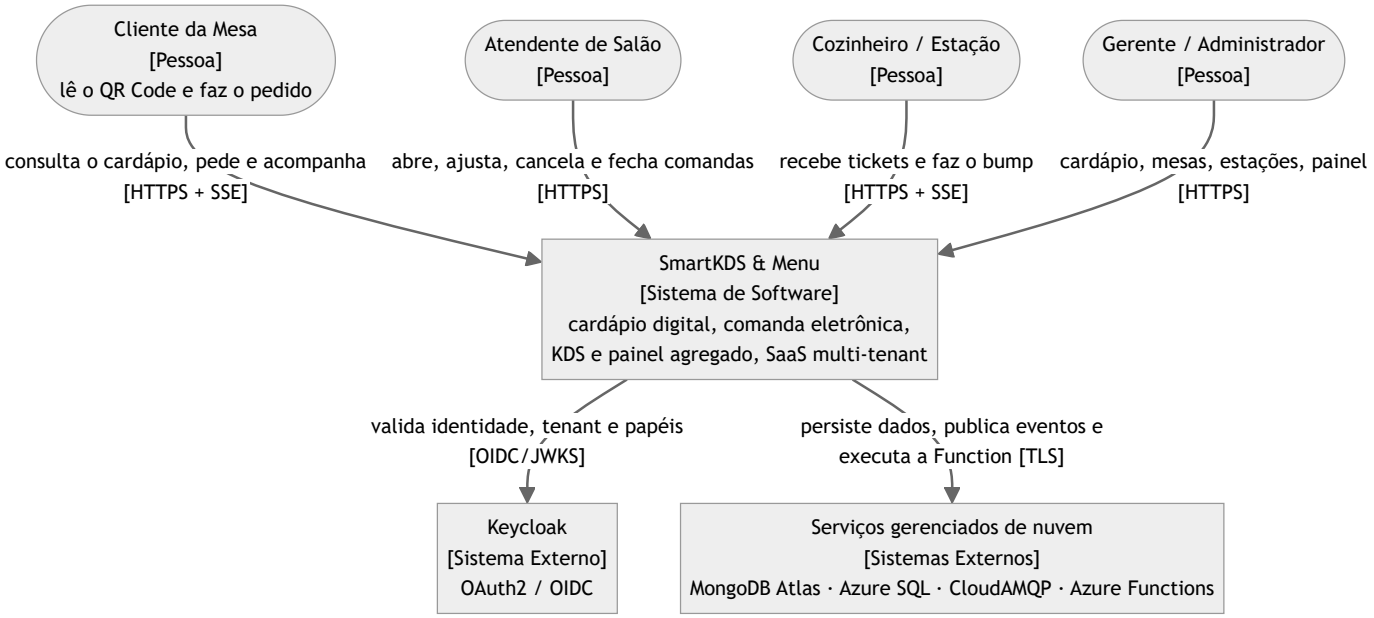


Figura E3-1: C4 Nível 1, Diagrama de Contexto (substitui a Figura 1)

Mudanças em relação à Figura 1: o armazenamento de imagens em S3 e o envio de e-mail ficaram fora do escopo da Entrega 3 (imagens são referenciadas por URL); os serviços gerenciados de nuvem passaram a ser explícitos.

4. C4 Nível 2 — Contêineres (atualizado)

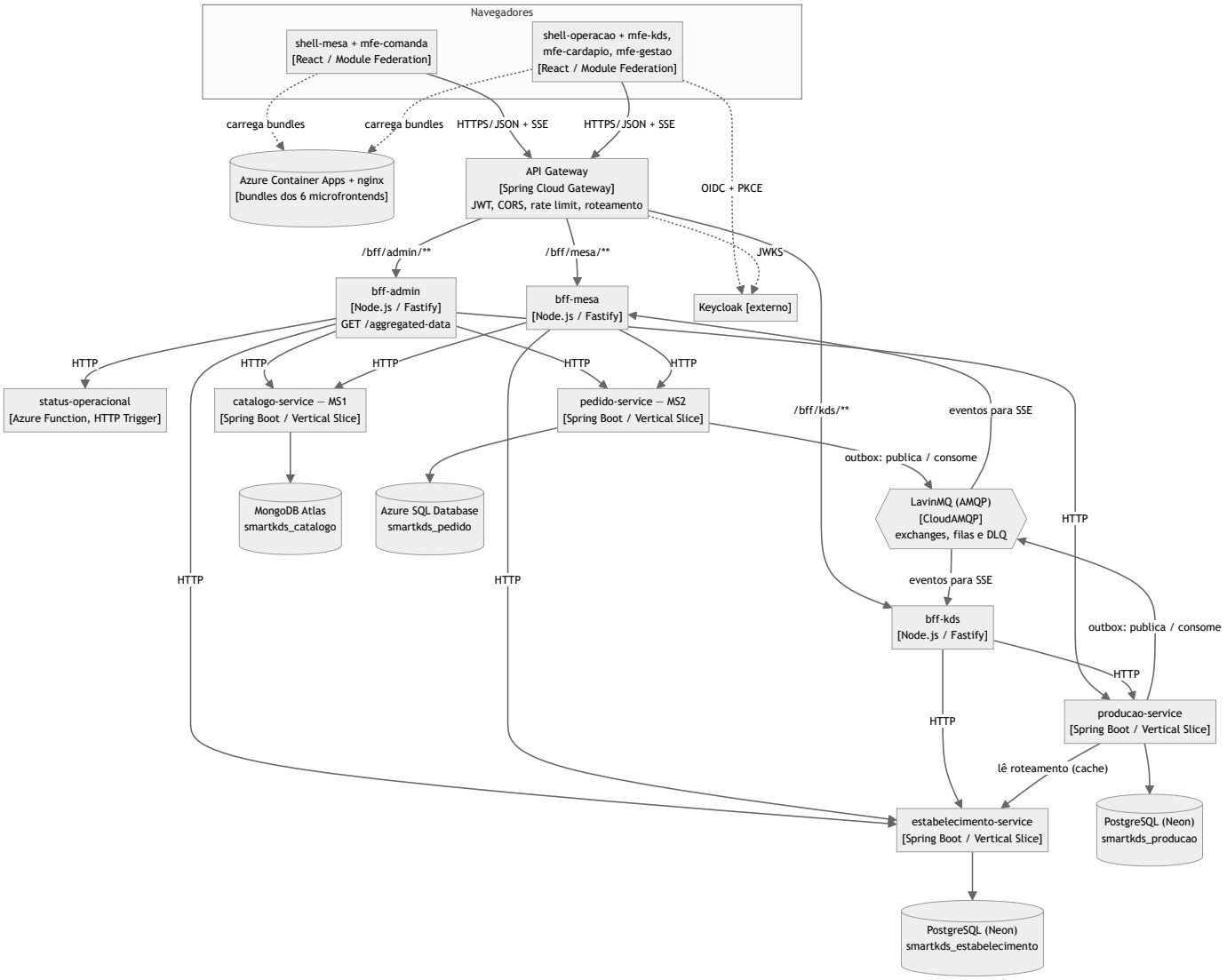


Figura E3-2: C4 Nível 2, Diagrama de Contêineres (substitui a Figura 2)

Contêiner	Tecnologia	Implantação	Imagem / artefato
Microfrontends (6)	React 18, Vite, Module Federation	Azure Container Apps com nginx (ingresso externo), um app por microfrontend	`smartkds-shell-*`, `smartkds-mfe-*`
API Gateway	Spring Cloud Gateway	Azure Container Apps (ingresso externo)	`smartkds-api-gateway`
bff-mesa, bff-kds, bff-admin	Node.js 22, Fastify	Container Apps (ingresso interno)	`smartkds-bff-*`
catalogo-service (MS1)	Spring Boot 3.5	Container Apps (interno) + MongoDB Atlas M0	`smartkds-catalogo-service`
pedido-service (MS2)	Spring Boot 3.5	Container Apps (interno) + Azure SQL Database	`smartkds-pedido-service`
producao-service, estabelecimento-service	Spring Boot 3.5	Container Apps (interno) + PostgreSQL 16 no Neon	`smartkds-producao-service`, `smartkds-estabelecimento-service`
status-operacional	Azure Functions v4 (Node 22)	Azure Functions (Consumption, Canada Central)	pacote da function (e `smartkds-status-operacional-function`)
Broker de eventos	LavinMQ (AMQP 0-9-1, compatível com RabbitMQ)	CloudAMQP (AWS São Paulo)	serviço gerenciado

5. C4 Nível 3 — Componentes (atualizado)

5.1 bff-admin: agregação e proxy CRUD

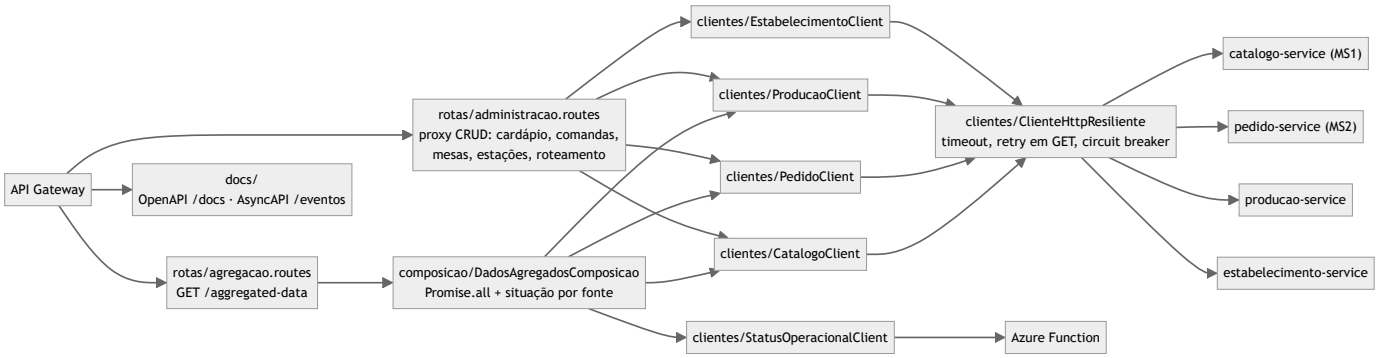


Figura E3-3: C4 Nível 3, Componentes do bff-admin

5.2 pedido-service (MS2): fatias verticais sobre o Azure SQL

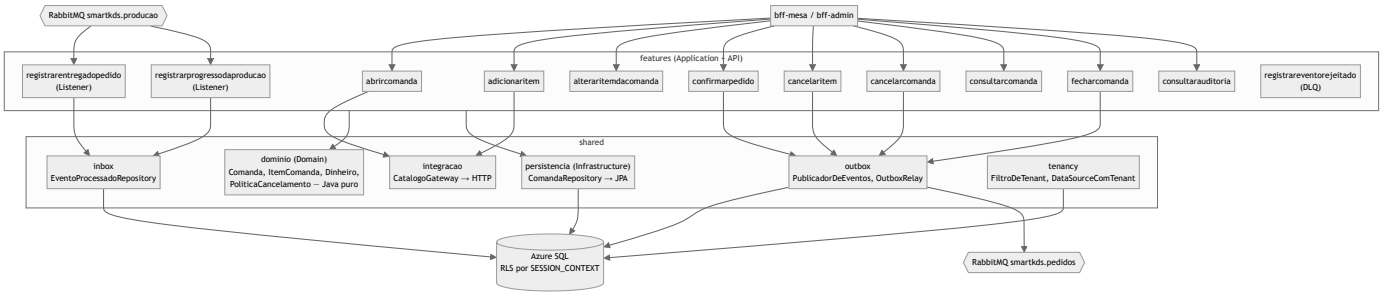


Figura E3-4: C4 Nível 3, Componentes do pedido-service (substitui a Figura 3)

5.3 catalogo-service (MS1): fatias verticais sobre o MongoDB Atlas

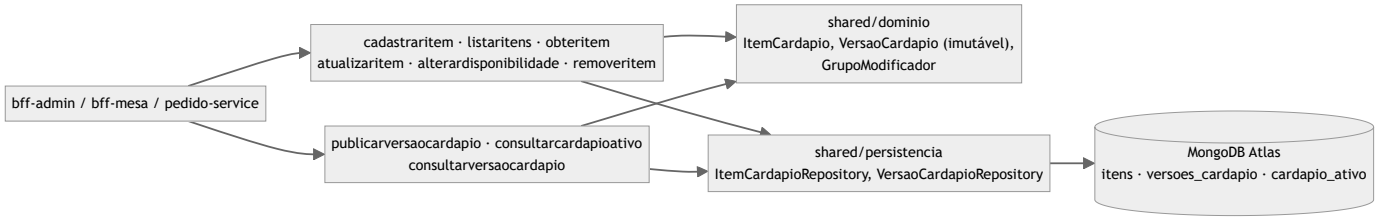


Figura E3-5: C4 Nível 3, Componentes do catalogo-service

6. Alterações nas seções do arc42

6.1 Seção 4.3 — bancos por serviço

Serviço	Banco	Justificativa
catalogo-service (MS1)	MongoDB Atlas (M0)	Documentos heterogêneos (modificadores) lidos inteiros; versão do cardápio como documento fechado.
pedido-service (MS2)	**Azure SQL Database**	Invariantes transacionais fortes e transação conjunta agregado + outbox; RLS nativo (`SECURITY POLICY`); índice filtrado para "uma comanda aberta por mesa"; `READPAST` para o relay. Migrações para PostgreSQL mantidas como alternativa.
producao-service	PostgreSQL	Consultas agregadas por estação e janela temporal (indicadores).
estabelecimento-service	PostgreSQL	Configuração pequena, relacional e muito consultada.

6.2 Seção 8.1.1 — DER do pedido-service no Azure SQL

Tipo lógico	PostgreSQL (Entrega 2)	Azure SQL (Entrega 3)
Identificadores	`UUID`	`UNIQUEIDENTIFIER`
Instantes	`TIMESTAMPTZ`	`DATETIMEOFFSET(6)`
Payload e detalhe JSON	`JSONB`	`NVARCHAR(MAX)` + `CHECK (ISJSON(...) = 1)`
Uma comanda aberta por mesa	índice único parcial	índice único filtrado `WHERE status = 'ABERTA'`
Row Level Security	`POLICY` + `current_setting('app.tenant_id')`	`SECURITY POLICY` + `SESSION_CONTEXT(N'tenant_id')`
Trava do relay da outbox	`FOR UPDATE SKIP LOCKED`	`WITH (UPDLOCK, ROWLOCK, READPAST)` (mesma consulta JPA)

6.3 Seção 7 — Implantação na nuvem

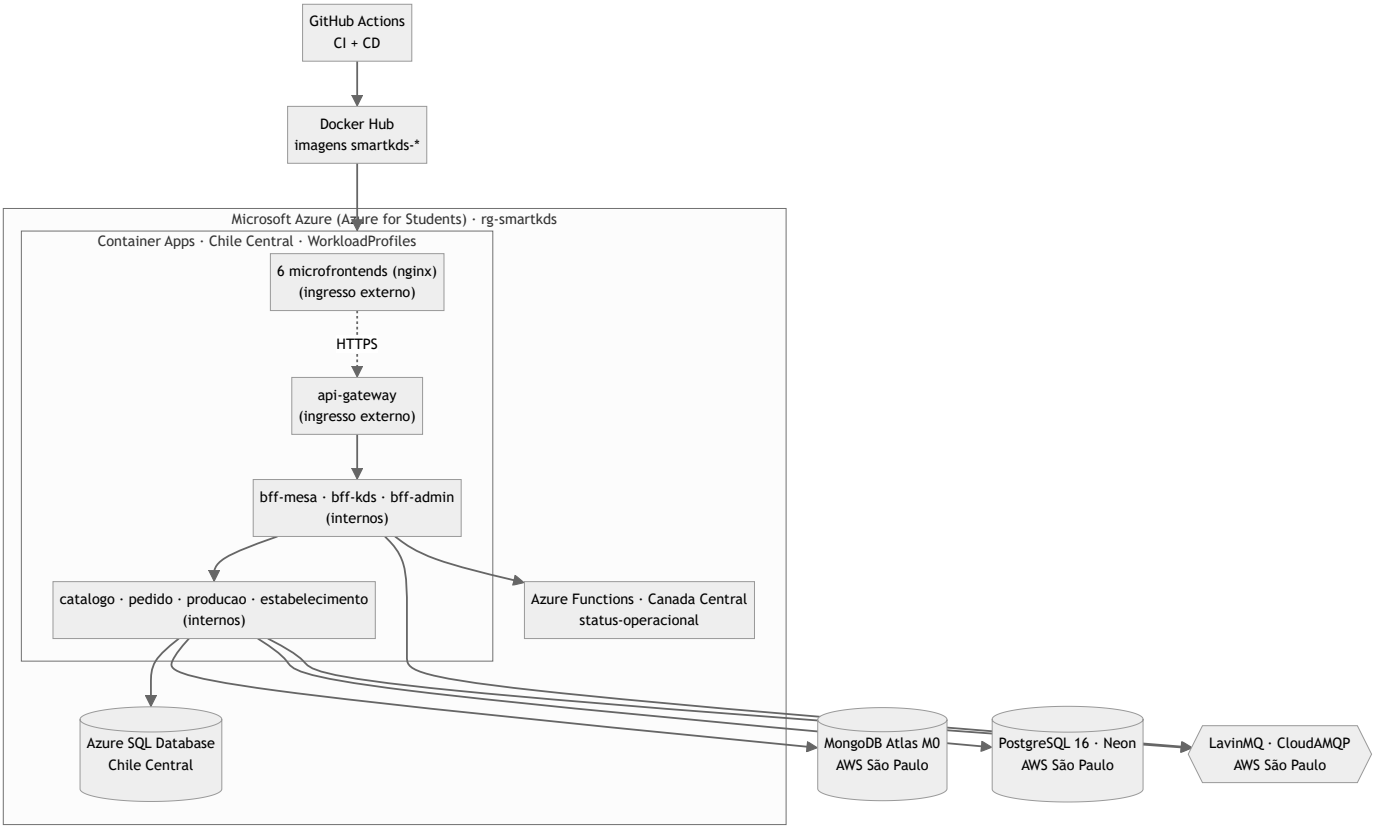


Figura E3-6: Implantação da Entrega 3

6.4 Decisões arquiteturais novas

ADR	Decisão	Motivo e consequência
ADR-13	pedido-service no **Azure SQL Database**, com migrações por fornecedor (`db/migration/{vendor}`)	Exigência da entrega e transação local agregado + outbox. A mesma camada JPA roda em Azure SQL e PostgreSQL; testes de contrato executam nos dois bancos (PostgreSQL embutido e SQL Server via Testcontainers).
ADR-14	**Azure Function** `status-operacional` (HTTP Trigger) para enriquecimento	Cálculo sem estado (faixa horária, pico, multiplicador) fora dos serviços de domínio; custo zero em repouso. A regra é pura e testada sem o SDK.
ADR-15	`GET /aggregated-data` no **bff-admin**	A agregação é composição de tela (papel do BFF, ADR-03). Chamadas em paralelo com circuit breaker; uma fonte fora não derruba a resposta.
ADR-16	Swagger dos microsserviços publicado pelo gateway em `/docs/<serviço>/**`	Contrato público sem expor `/interno/**` ao navegador; o gateway calcula `X-Forwarded-Prefix` (trusted proxies).
ADR-17	**Monorepo** público com projetos independentes	Cada projeto (Microfrontend, BFF, MS Mongo, MS SQL, Function, gateway) fica em pasta própria com README, Dockerfile, build, testes e job de CI isolados (matriz do GitHub Actions) e imagem própria no Docker Hub, preservando o deploy independente. Um único repositório mantém contratos (AsyncAPI), docker-compose, infra e documentação versionados juntos e simplifica a avaliação. O script `scripts/publicar-github.sh` permite separar em um repositório por projeto, se necessário.
ADR-18	CRUD completo da comanda: alterar item do carrinho e cancelar comanda (exclusão lógica)	Cancelar comanda com itens já na cozinha gera `ItemCancelado` para cada um (mesma saga da Figura 19); recusado se algo já estiver em produção.
ADR-19	API Gateway com **Spring Cloud Gateway** em vez do AWS API Gateway	O enunciado permite outro gateway. Toda a solução roda na Azure (Container Apps); um gateway AWS exigiria uma segunda nuvem, latência entre provedores e outra conta. O Spring Cloud Gateway cobre roteamento, segurança (JWT/Keycloak, tenant sobrescrito, CORS, rate limit) e entrada única, com testes de borda e ArchUnit.

ADR-20	Regiões e hospedagem impostas pela assinatura **Azure for Students**	A política da assinatura só permite 5 regiões (southafricanorth, canadacentral, northcentralus, spaincentral, chilecentral). Container Apps e Azure SQL ficam em **Chile Central** (a mais próxima dos usuários); a Function fica em **Canada Central** (Chile Central não tem plano de consumo). O Static Web Apps não existe em nenhuma região permitida, então os 6 microfrontends rodam como Container Apps com nginx (workflow `cd-frontend.yml`). O ambiente usa o modo **WorkloadProfiles**: o modo Express não aplica o ingresso interno nem resolve o nome curto dos apps. Oferta gratuita do Azure SQL indisponível na região: plano Basic.
ADR-21	Broker **LavinMQ** (CloudAMQP) e PostgreSQL no **Neon**	O plano gratuito da CloudAMQP passou a oferecer LavinMQ, que implementa o mesmo protocolo AMQP 0-9-1: exchanges topic, DLX, filas exclusivas e publisher confirms foram validados sem mudar código. Neon (PostgreSQL 16, São Paulo) evita gastar o crédito estudantil; um usuário não superusuário por banco mantém o RLS efetivo. Mongo, Neon e LavinMQ ficam todos em São Paulo.

6.5 Outros ajustes da implementação

Tema	Documento anterior	Implementado
Composição dos handlers	`HandlerConfig` com handlers package-private	Handlers anotados com `@Component` (continuam package-private)
Ponteiro da versão ativa	no estabelecimento e no catálogo	somente no catálogo (`cardapio_ativo`)
Roteamento por categoria	evento sem categoria	`PedidoConfirmado` carrega `itemCardapioId` e `categoria`
Filas	uma fila por serviço	uma fila por tipo de evento consumido; filas de SSE exclusivas por réplica
Rate limit	Redis	token bucket em memória por réplica (Redis como evolução)
Token no SSE	—	`?access_token=` aceito apenas pelo gateway
Integrantes	tabela com 2	4 integrantes, como na capa

6.6 Testes (Seção 8.8.6)

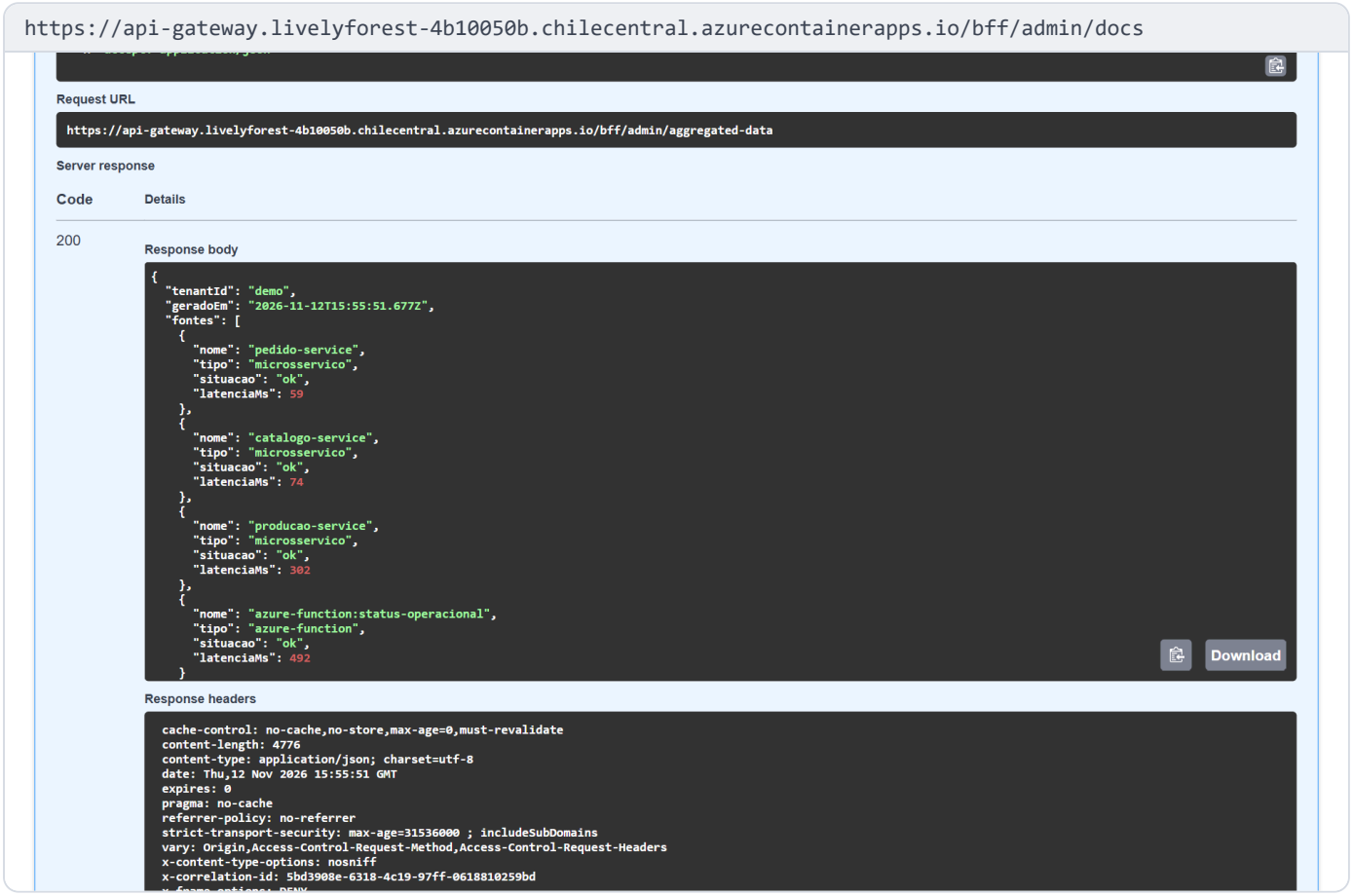
Projeto	Testes	Tipos
catalogo-service	19	domínio, fatia, 13 regras ArchUnit
pedido-service	52	domínio (CRUD da comanda), fatias, 18 regras ArchUnit, integração PostgreSQL e Azure SQL (Testcontainers), trava concorrente da outbox
producao-service	28	domínio, roteamento, idempotência, ArchUnit, integração (indicadores e RLS)
estabelecimento-service	20	domínio, ArchUnit, integração
api-gateway	7	filtros de borda, rate limit, ArchUnit
BFFs	37	composição, rotas, circuit breaker, SSE, `/aggregated-data`, cache (QR Code revogado não é servido), arquitetura
Azure Function	6	regra pura e arquitetura
Microfrontends	28	componentes, regras de UI e arquitetura

7. Prints

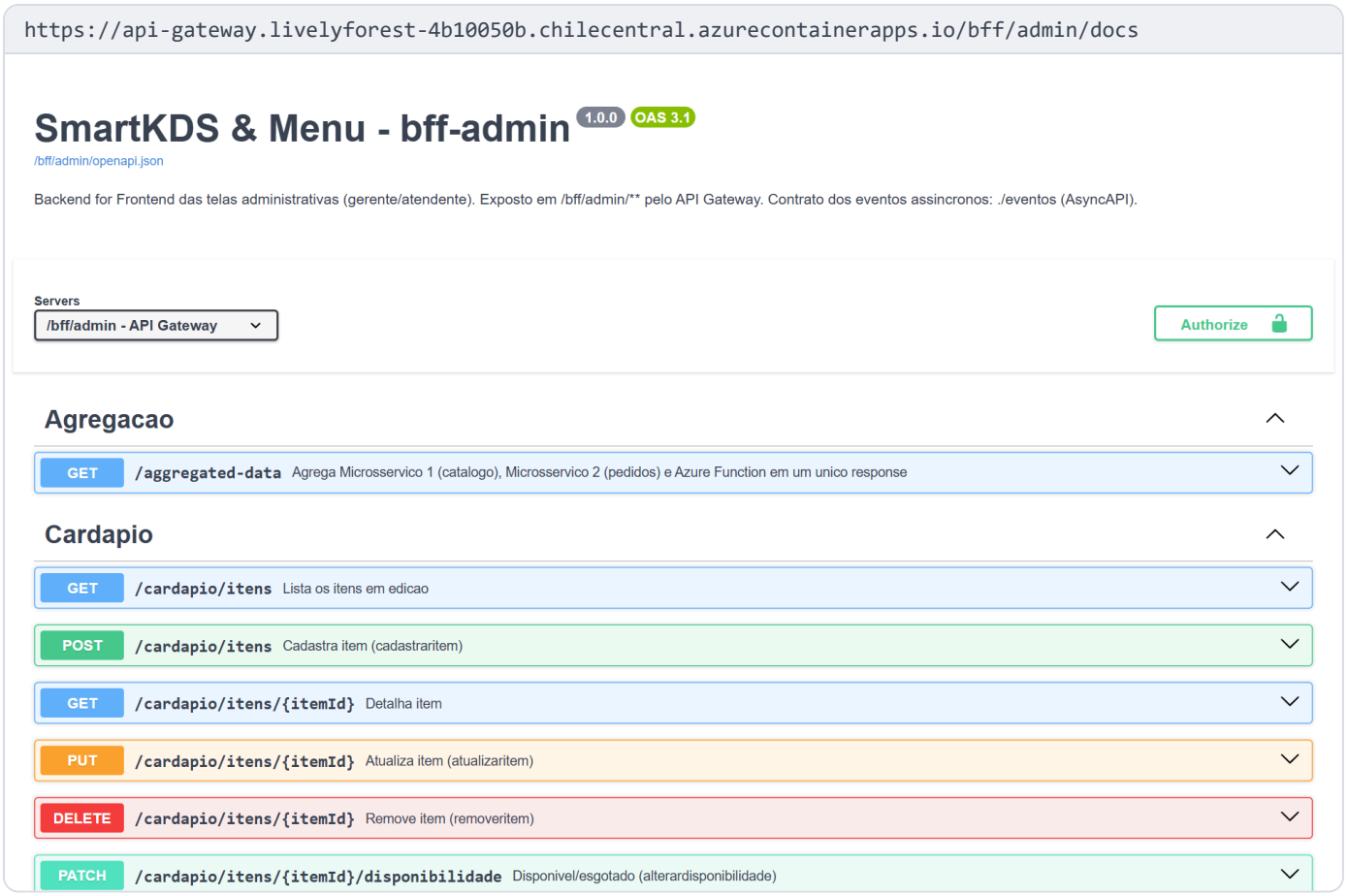
> Prints capturados em 12/11/2026 com o sistema publicado na Azure, todas as chamadas pelo API Gateway público, depois de `scripts/popular-demo.sh` e de um pedido real feito pela Mesa 2. Acima de cada print está o endereço aberto. Os prints 19 e 20 reproduzem a saída real de `curl` e da Azure CLI executados contra a nuvem.

7.1 Swagger dos BFFs

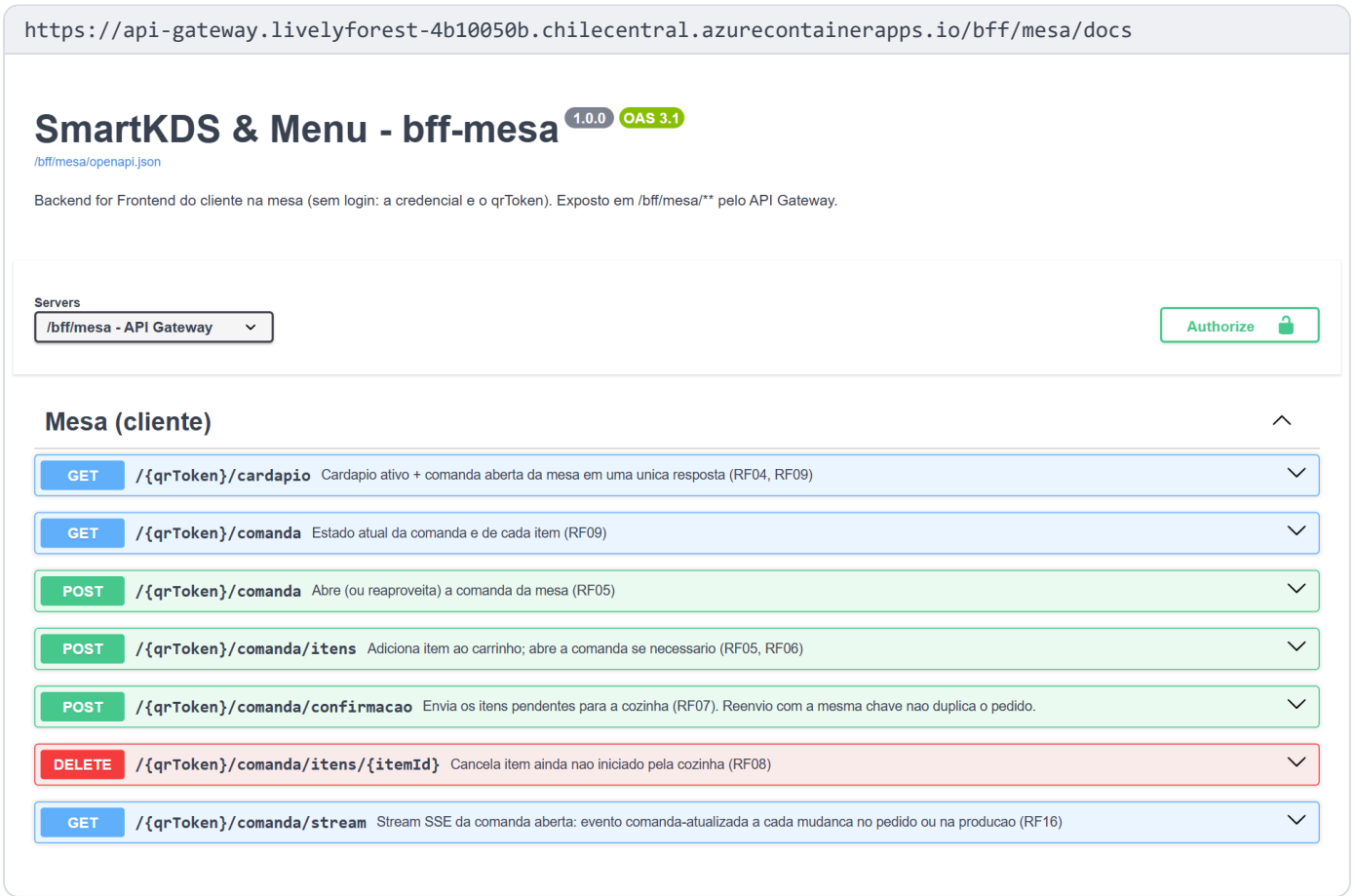
`https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/admin/docs`



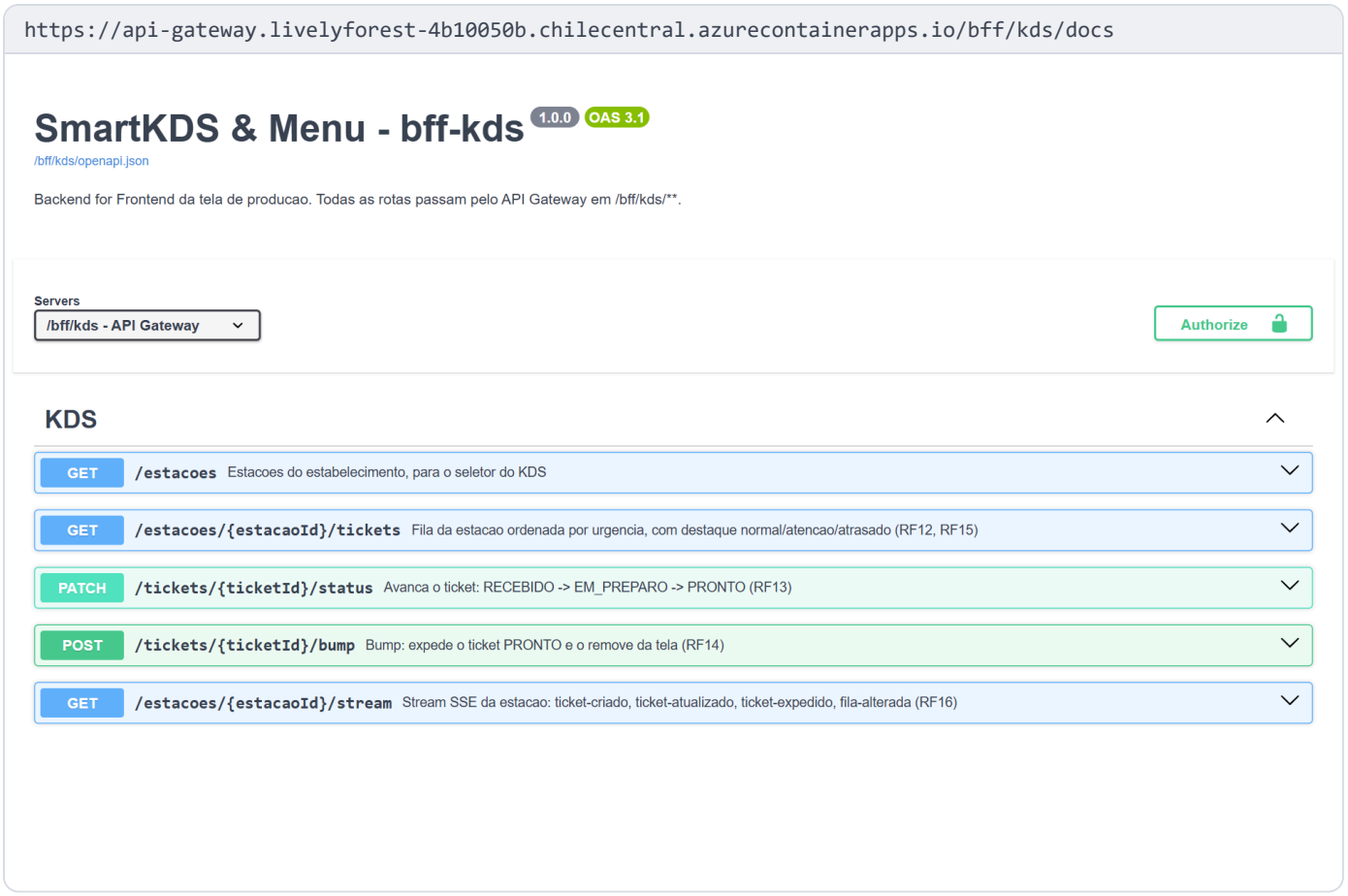
Print 1 — Swagger do bff-admin: `GET /aggregated-data` executado pelo gateway da nuvem, com as quatro fontes (pedido, catálogo, produção e Azure Function) em situação "ok"



Print 2 — Swagger do bff-admin: agregação e proxy CRUD (cardápio, comandas, mesas, estações, roteamento)



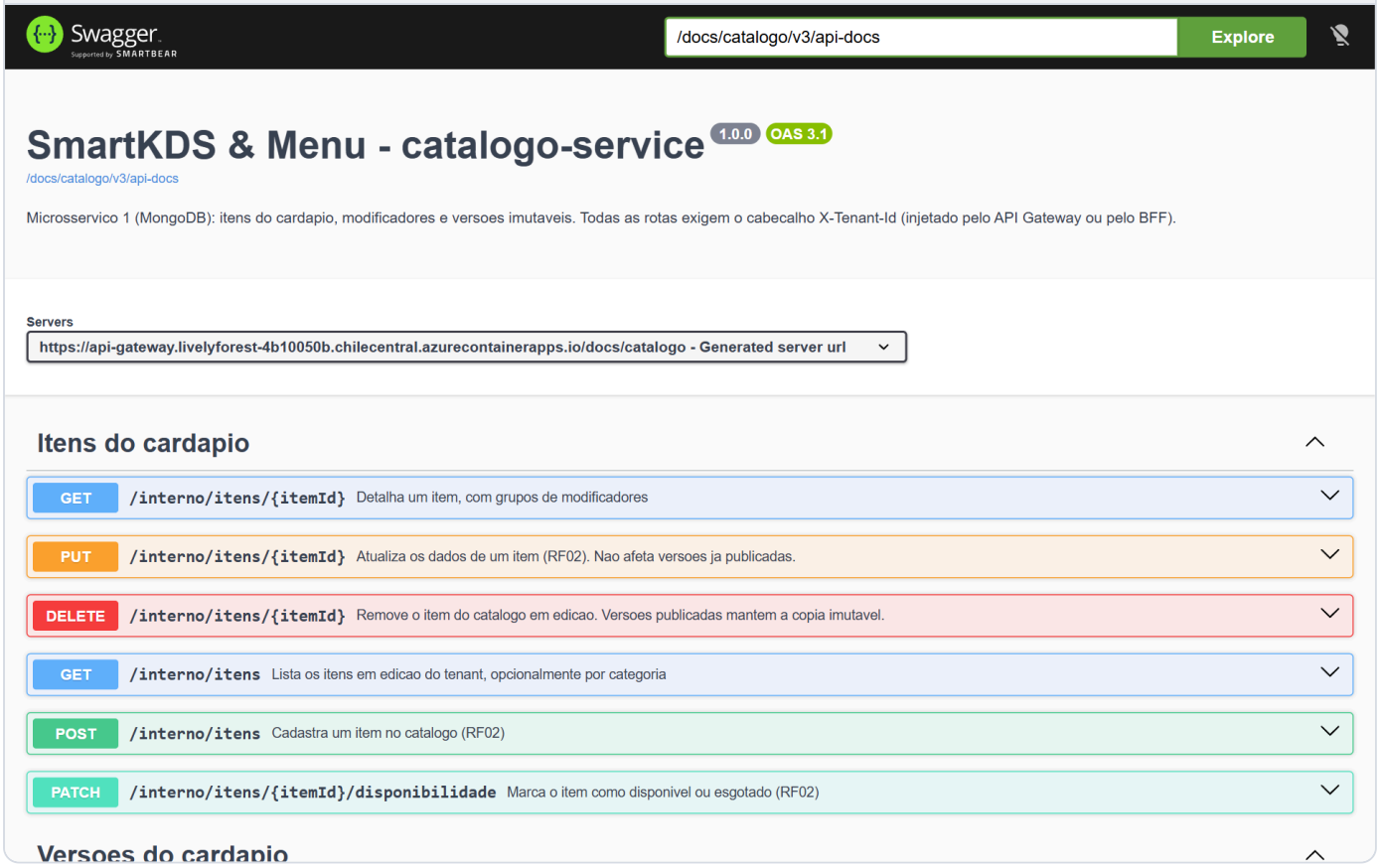
Print 3 — Swagger do bff-mesa (experiência do cliente pelo QR Code)



Print 4 — Swagger do bff-kds (fila por estação, status, bump e stream SSE)

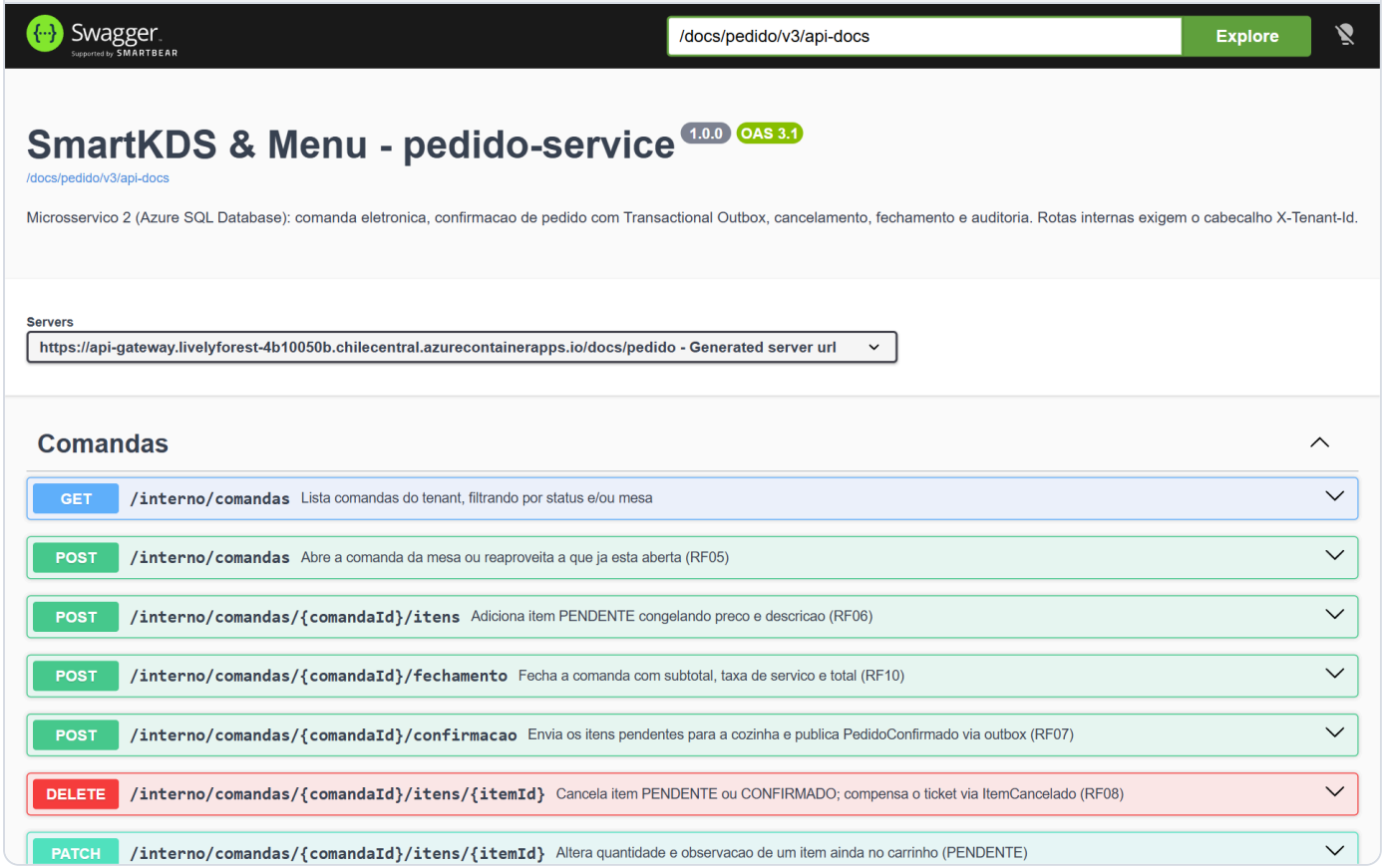
7.2 Swagger dos microsserviços

https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/docs/catalogo/swagger-ui/index.html

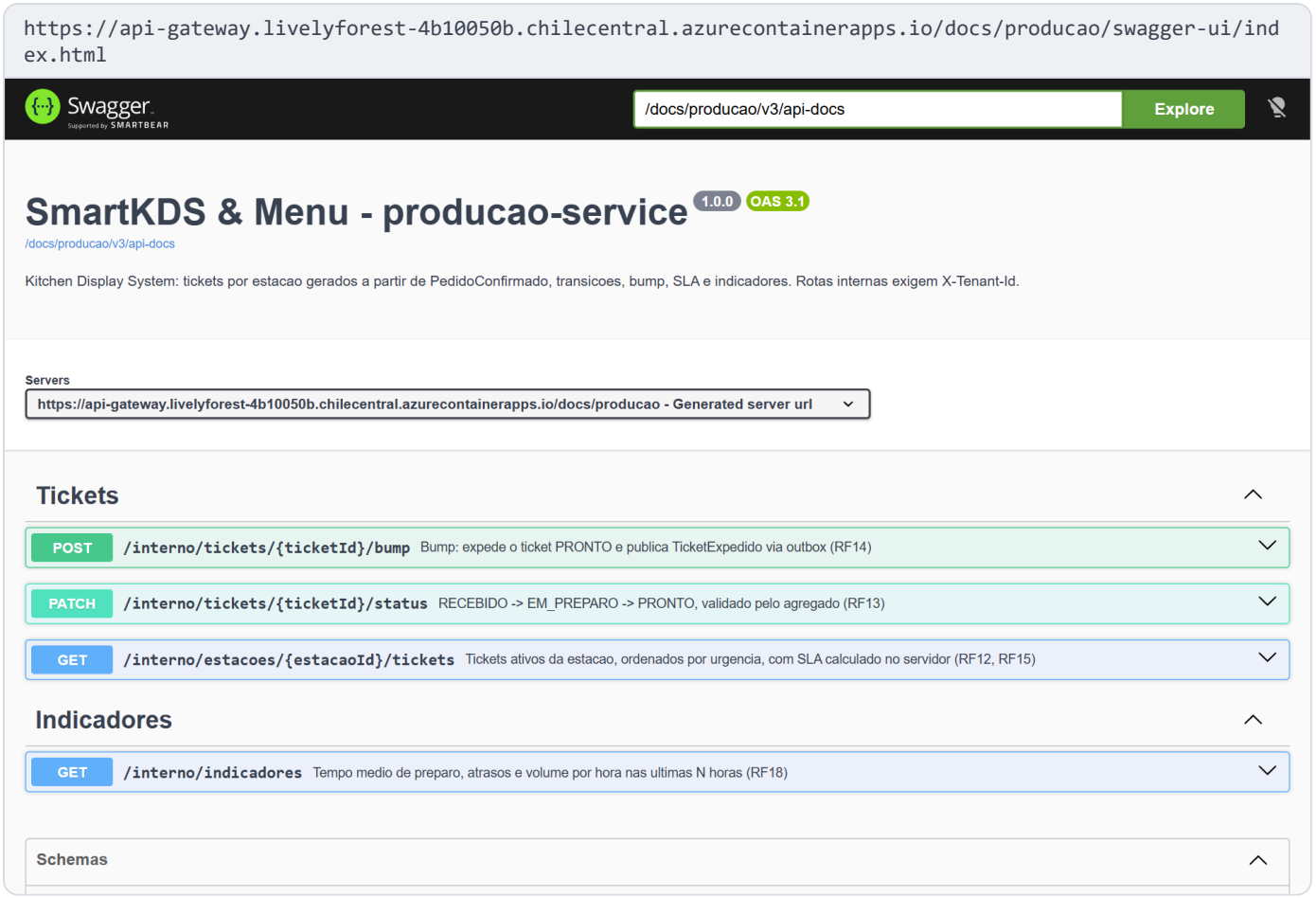


Print 5 — Microsserviço 1 (MongoDB Atlas): catalogo-service, CRUD de itens e versões do cardápio

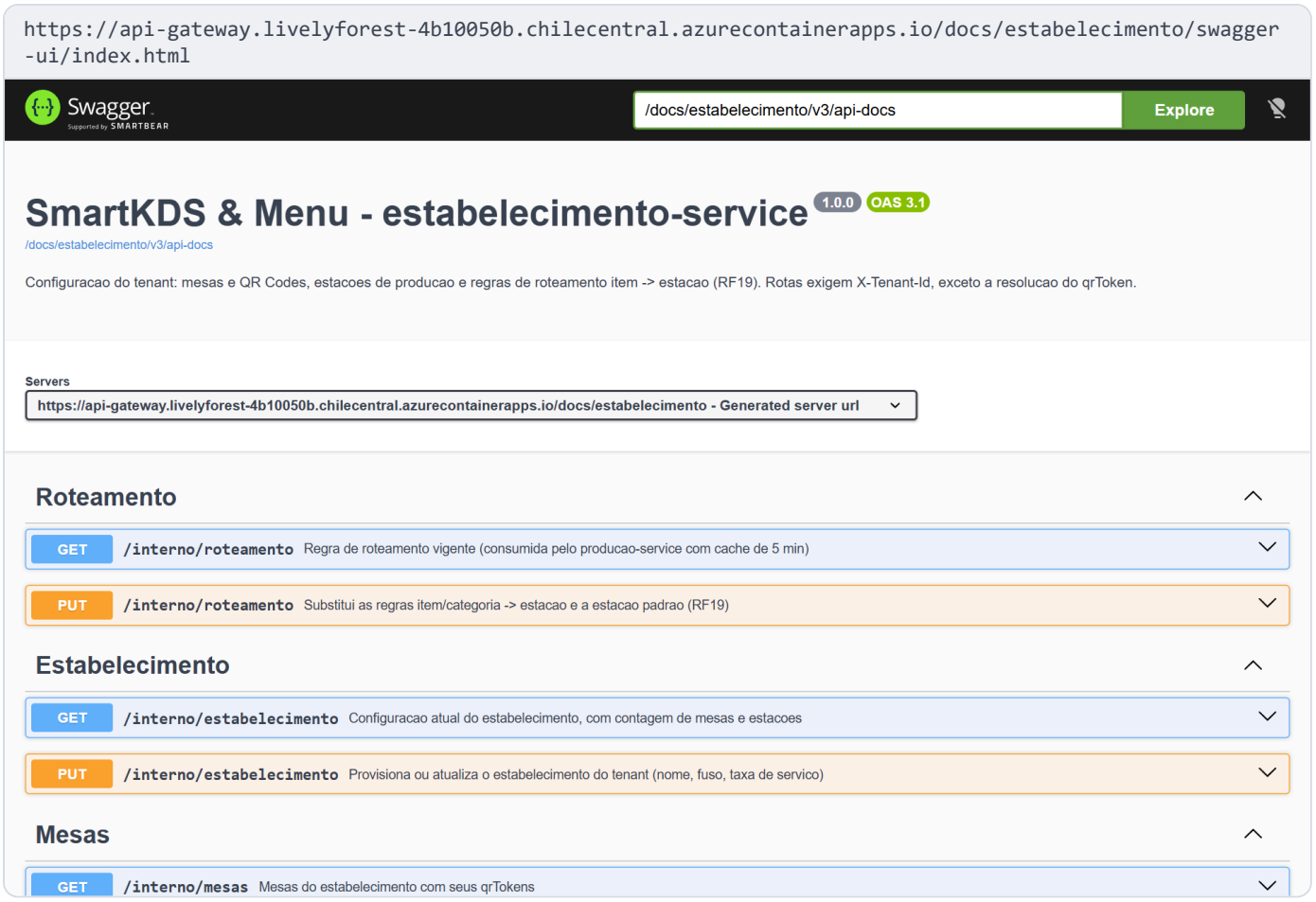
https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/docs/pedido/swagger-ui/index.html



Print 6 — Microsserviço 2 (Azure SQL): pedido-service, POST/GET/PATCH/DELETE de comandas e itens

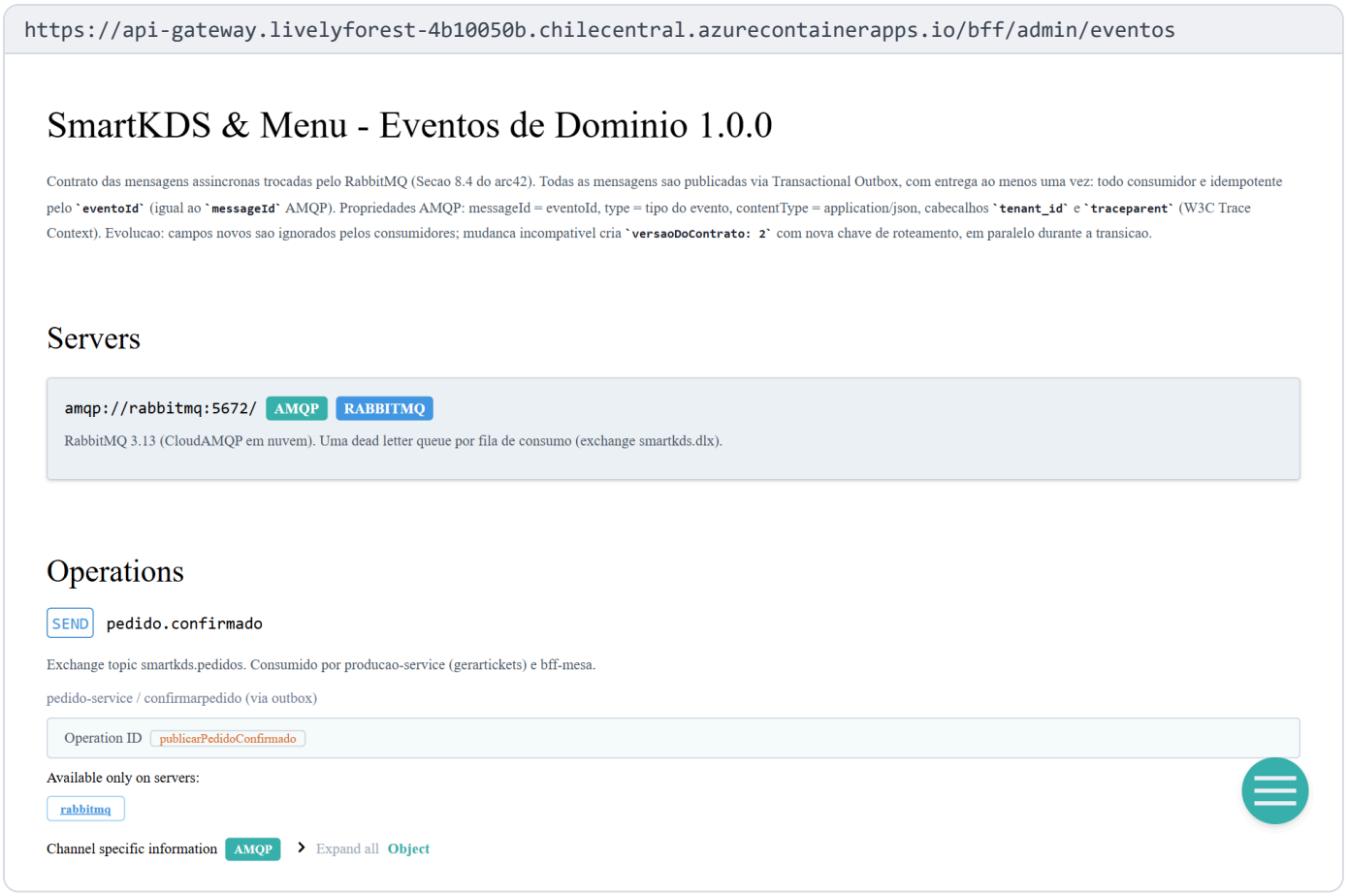


Print 7 — producao-service (tickets, status, bump e indicadores)



Print 8 — estabelecimento-service (estabelecimento, mesas, estações e roteamento)

7.3 Documentação dos eventos (AsyncAPI)



Print 9 — AsyncAPI 3.0 dos eventos de domínio publicada pelo bff-admin, com a operação `pedido.confirmado` (o broker em produção é o LavinMQ, que fala o mesmo protocolo AMQP)

7.4 Frontend em funcionamento



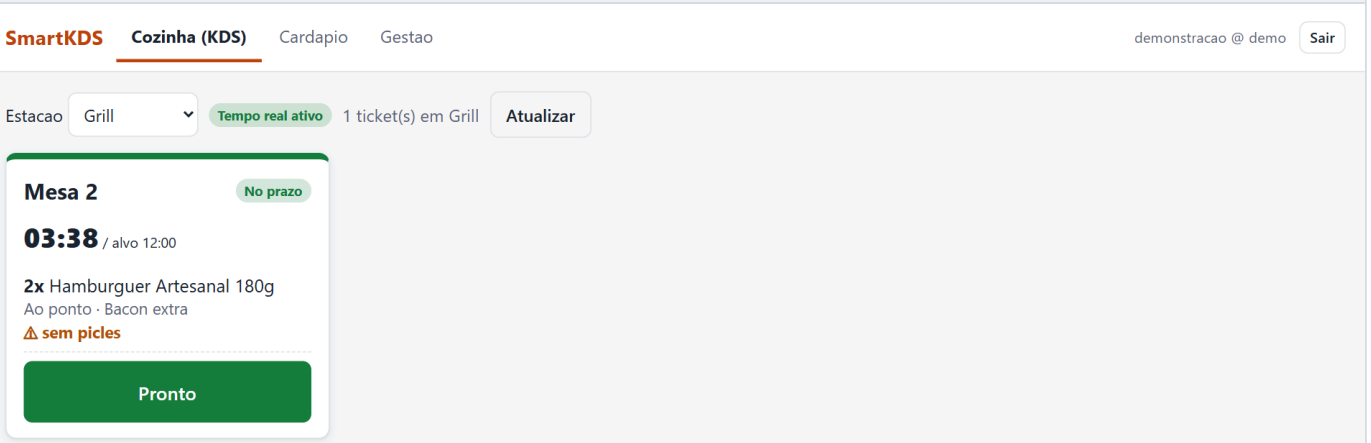
Print 10 — shell-mesa + mfe-comanda no celular: cardápio da Mesa 2 e comanda com um item "Em preparo" (atualizado em tempo real pelo KDS), itens enviados à cozinha e um item no carrinho

https://shell-operacao.livelyforest-4b10050b.chilecentral.azurecontainerapps.io

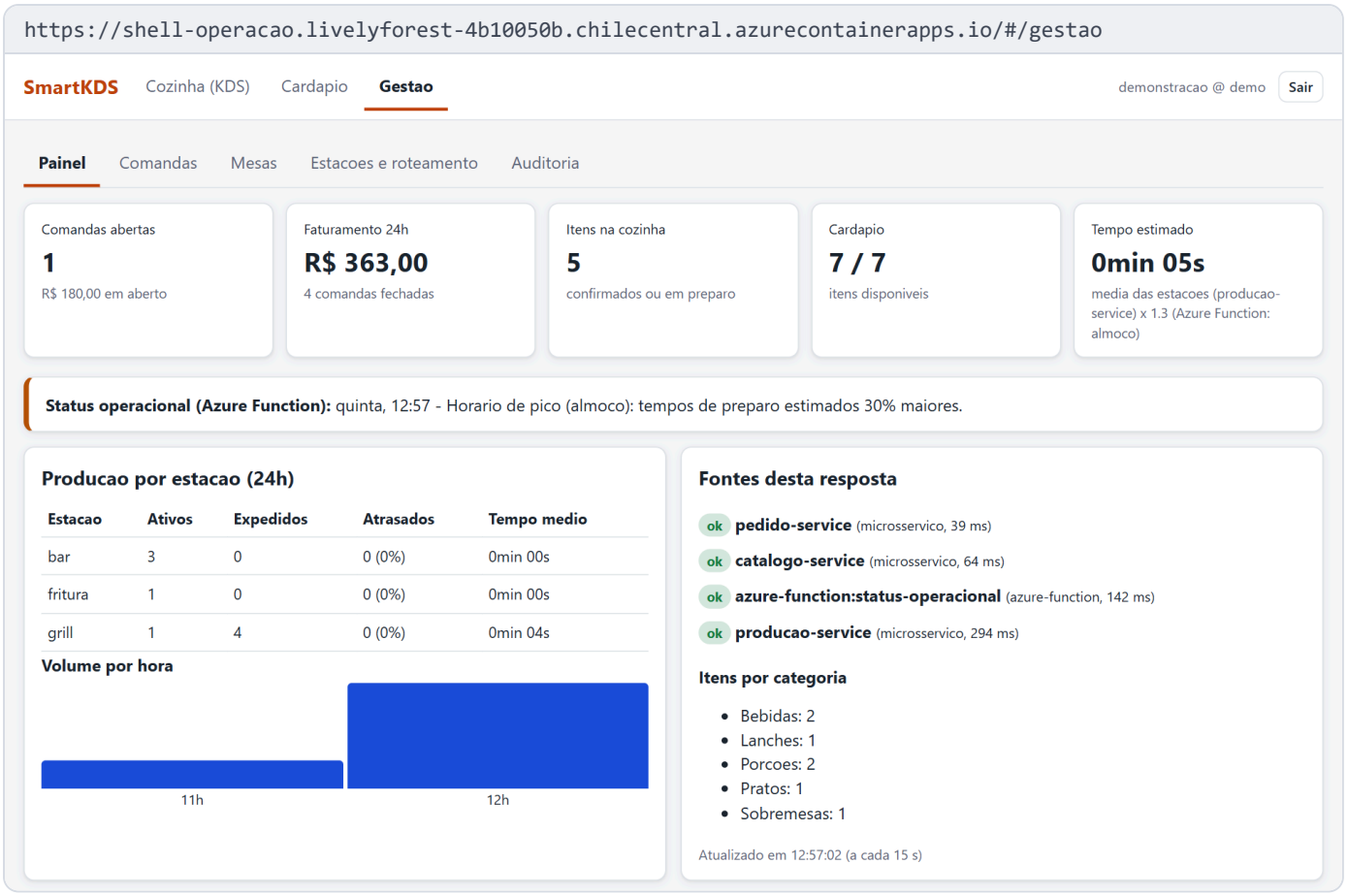


Print 11 — shell-operacao: entrada (modo demonstração; com Keycloak, login OIDC + PKCE)

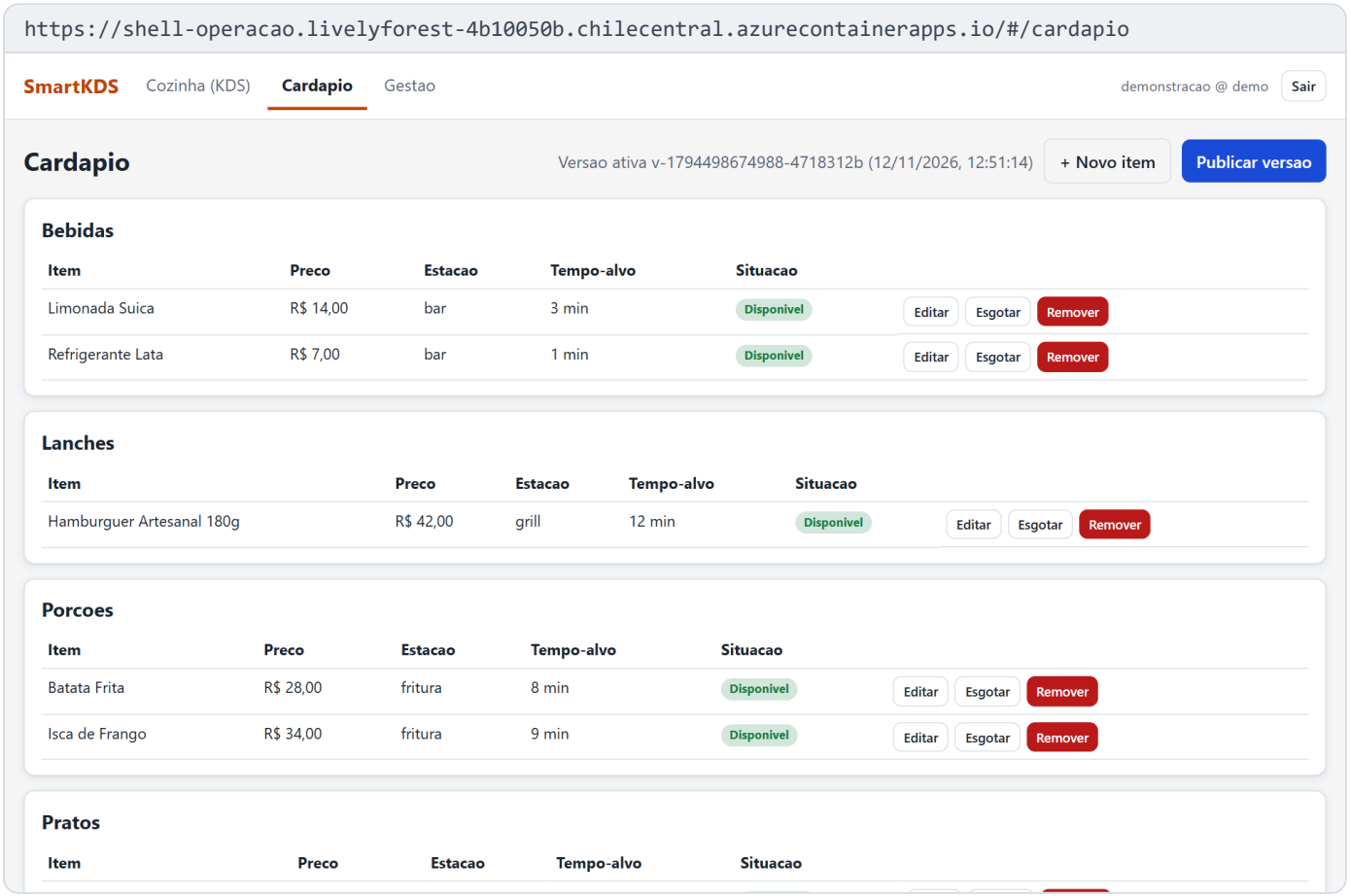
https://shell-operacao.livelyforest-4b10050b.chilecentral.azurecontainerapps.io



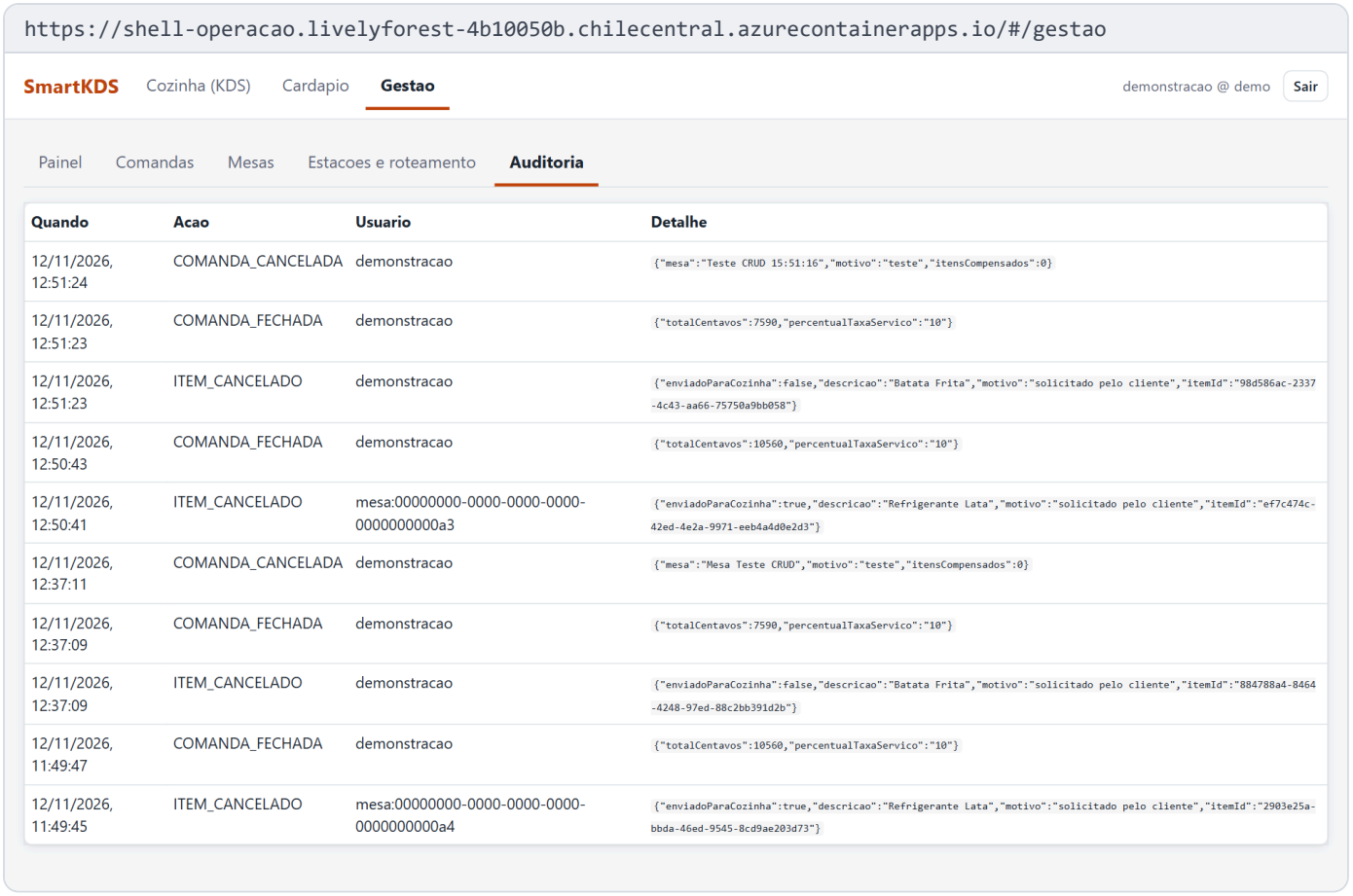
Print 12 — mfe-kds: estação Grill com o ticket da Mesa 2 em preparo, cronômetro, SLA e "Tempo real ativo" (SSE)



Print 13 — mfe-gestao: painel do gerente consumindo `GET /aggregated-data` (MS1 + MS2 + Azure Function + produção), com o status de pico calculado pela Function



Print 14 — mfe-cardapio: manutenção do cardápio (CRUD via bff-admin → catalogo-service → MongoDB Atlas)



Print 15 — mfe-gestao, aba Auditoria: trilha gravada pelo pedido-service no Azure SQL (fechamentos e cancelamentos de comanda)

7.5 API Gateway, endpoint agregado e Azure Function

https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/admin/aggregated-data

Estilos de formatação

{"tenantId":"demo","geradoEm":"2026-11-12T15:57:30.501Z","fontes":[{"nome":"pedido-service","tipo":"microsservico","situacao":"ok","latenciaMs":54},{"nome":"catalogo-service","tipo":"microsservico","situacao":"ok","latenciaMs":76},{"nome":"producao-service","tipo":"microsservico","situacao":"ok","latenciaMs":368},{"nome":"azure-function:status-operacional","tipo":"azure-function","situacao":"ok","latenciaMs":597}],"catalogo":
{"totalDeItens":7,"itensDisponiveis":7,"itensEsgotados":0,"precoMedioCentavos":3429,"porCategoria":[{"categoria":"Bebidas","itens":2},{"categoria":"Lanches","itens":1},
{"categoria":"Porcoes","itens":2},{"categoria":"Pratos","itens":1},{"categoria":"Sobremesas","itens":1}],"itens":[{"id":"e2f35cfb-697f-4028-bf74-37fdb6859d93","nome":"Limonada Suica","categoria":"Bebidas","precoCentavos":1400,"estacaoSugerida":"bar","tempoAlvoPreparoSegundos":180,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:29.007Z"},
{"id":"449e2610-0e6c-49b0-8bbb-25828e53435e","nome":"Refrigerante Lata","categoria":"Bebidas","precoCentavos":700,"estacaoSugerida":"bar","tempoAlvoPreparoSegundos":60,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:29.472Z"},{"id":"3aa407b7-f6e9-44ae-a3b7-089447e27014","nome":"Hamburguer Artesanal 180g","categoria":"Lanches","precoCentavos":4200,"estacaoSugerida":"grill","tempoAlvoPreparoSegundos":720,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:26.408Z"},
{"id":"d37353b7-c7e1-482b-a375-0e1338b59e33","nome":"Batata Frita","categoria":"Porcoes","precoCentavos":2800,"estacaoSugerida":"fritura","tempoAlvoPreparoSegundos":480,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:27.940Z"},
{"id":"8d17e03a-8fd7-4b42-91d0-2d873852e0ea","nome":"Isca de Frango","categoria":"Porcoes","precoCentavos":3400,"estacaoSugerida":"fritura","tempoAlvoPreparoSegundos":540,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:28.563Z"},
{"id":"2ec3f620-dd46-4a81-b80b-9587b8ebd920","nome":"Picanha na Chapa","categoria":"Pratos","precoCentavos":8900,"estacaoSugerida":"grill","tempoAlvoPreparoSegundos":900,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:27.433Z"},
{"id":"0b4918bf-971e-4c89-b961-60f0bdc34bc6","nome":"Petit Gateau","categoria":"Sobremesas","precoCentavos":2600,"estacaoSugerida":"sobremesa","tempoAlvoPreparoSegundos":420,"disponivel":true,"atualizadoEm":"2026-11-12T14:49:29.943Z"}]},"pedidos":
{"comandasAbertas":1,"comandasFechadasNasUltimas24h":4,"valorEmAbertoCentavos":18000,"faturamentoNasUltimas24hCentavos":36300,"itensAguardandoCozinha":5,"comandasRecentes":
[{"comandaId":"40401fad-45c9-4c8e-bd9c-6345dbe5b7cc","mesaRotulo":"Mesa 2","status":"ABERTA","subtotalCentavos":18000,"abertaEm":"2026-11-12T15:53:17.703422Z"},
{"comandaId":"663a95f9-0d55-47f9-bef5-fda1b36892c7","mesaRotulo":"Teste CRUD 15:51:16","status":"CANCELADA","subtotalCentavos":0,"abertaEm":"2026-11-12T15:51:24.261819Z"},
{"comandaId":"78011158-c387-44a9-b488-3ee01a193188","mesaRotulo":"Teste CRUD 15:51:16","status":"FECHADA","subtotalCentavos":6900,"abertaEm":"2026-11-12T15:51:20.912223Z"},
{"comandaId":"3cc9d108-6fa2-492a-9463-a3b5cfaf93c4","mesaRotulo":"Mesa 3","status":"FECHADA","subtotalCentavos":9600,"abertaEm":"2026-11-12T15:50:34.095099Z"},
{"comandaId":"3df44988-6ba7-4ab8-99c0-b03f0bc0f2cf","mesaRotulo":"Mesa Teste CRUD","status":"CANCELADA","subtotalCentavos":0,"abertaEm":"2026-11-12T15:37:10.776914Z"},
{"comandaId":"2c730077-524d-469a-aebc-c8631e77f342","mesaRotulo":"Mesa Teste CRUD","status":"FECHADA","subtotalCentavos":6900,"abertaEm":"2026-11-12T15:37:06.729084Z"},
{"comandaId":"9b42f9a6-0b25-4b94-aaee-489c701bc0d9","mesaRotulo":"Mesa 4","status":"FECHADA","subtotalCentavos":9600,"abertaEm":"2026-11-12T14:49:36.280989Z"}]},"statusOperacional":
{"tenantId":"demo","fusoHorario":"America/Sao_Paulo","horaLocal":"12:57","diaDaSemana":"quinta","faixaHoraria":"almoco","emHorarioDePico":true,"multiplicadorTempoPreparo":1.3,"mensagem":"Horario de pico (almoco): tempos de preparo estimados 30% maiores.","geradoEm":"2026-11-12T15:57:30.428Z"},"producao":{"desde":"2026-11-11T15:57:29.914701894Z","geradoEm":"2026-11-12T15:57:29.914701894Z","estacoes":
[{"estacaoId":"bar","ticketsCriados":5,"ticketsExpedidos":0,"ticketsAtivos":3,"ticketsAtrasados":0,"taxaDeAtraso":0,"tempoMedioPreparoSegundos":0},
{"estacaoId":"fritura","ticketsCriados":1,"ticketsExpedidos":0,"ticketsAtivos":1,"ticketsAtrasados":0,"taxaDeAtraso":0,"tempoMedioPreparoSegundos":0},
{"estacaoId":"grill","ticketsCriados":5,"ticketsExpedidos":4,"ticketsAtivos":1,"ticketsAtrasados":0,"taxaDeAtraso":0,"tempoMedioPreparoSegundos":4}],"volumePorHora":
[{"hora":11,"tickets":2},{"hora":12,"tickets":9}]},"estimativa":{"tempoMedioPreparoSegundos":4,"tempoEstimadoComPicoSegundos":5,"explicacao":"media das estacoes (producao-service) x 1.3 (Azure Function: almoco)"}}

Print 16 — Resposta do endpoint agregado servida pelo gateway público

https://smartkds-status-5560.azurewebsites.net/api/status-operacional?tenantId=demo

Estilos de formatação

{"tenantId":"demo","fusoHorario":"America/Sao_Paulo","horaLocal":"12:57","diaDaSemana":"quinta","faixaHoraria":"almoco","emHorarioDePico":true,"multiplicadorTempoPreparo":1.3,"mensagem":"Horario de pico (almoco): tempos de preparo estimados 30% maiores.","geradoEm":"2026-11-12T15:57:34.225Z"}

Print 17 — Azure Function `status-operacional` (HTTP Trigger) respondendo na nuvem

Terminal — API Gateway SmartKDS (Azure Container Apps)

```
$ curl -si https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/kds/estacoes/grill/tickets -H 'X-Tenant-Id: demo' | head -7
HTTP/1.1 200 OK
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-correlation-id: 6ac9aaa9-dddd-4b3a-9a8f-c123f209facf
content-type: application/json; charset=utf-8
content-length: 119
date: Thu, 12 Nov 2026 15:52:29 GMT
cache-control: no-cache, no-store, max-age=0, must-revalidate

$ curl -si -X OPTIONS https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/bff/admin/aggregated-data -H 'Origin: https://shell-operacao.livelyforest-4b10050
b.chilecentral.azurecontainerapps.io' -H 'Access-Control-Request-Method: GET' | head -8
HTTP/1.1 200 OK
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
access-control-allow-origin: https://shell-operacao.livelyforest-4b10050b.chilecentral.azurecontainerapps.io
access-control-allow-methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
access-control-expose-headers: X-Correlation-Id
access-control-max-age: 3600
cache-control: no-cache, no-store, max-age=0, must-revalidate
pragma: no-cache

$ curl -s -o /dev/null -w '%{http_code}' https://api-gateway.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/interno/comandas      # rota interna nao exposta
404

$ curl -s -o /dev/null -w '%{http_code}' https://pedido-service.internal.livelyforest-4b10050b.chilecentral.azurecontainerapps.io/   # servico interno pela internet
404
```

Print 18 — API Gateway: roteamento para o bff-kds com `X-Correlation-Id` gerado na borda, CORS liberado para o frontend publicado, `/interno/**` bloqueado (404) e serviço interno inacessível pela internet (404)

7.6 Publicação: Azure, Docker Hub e GitHub Actions

Terminal — Azure CLI, grupo de recursos rg-smartkds

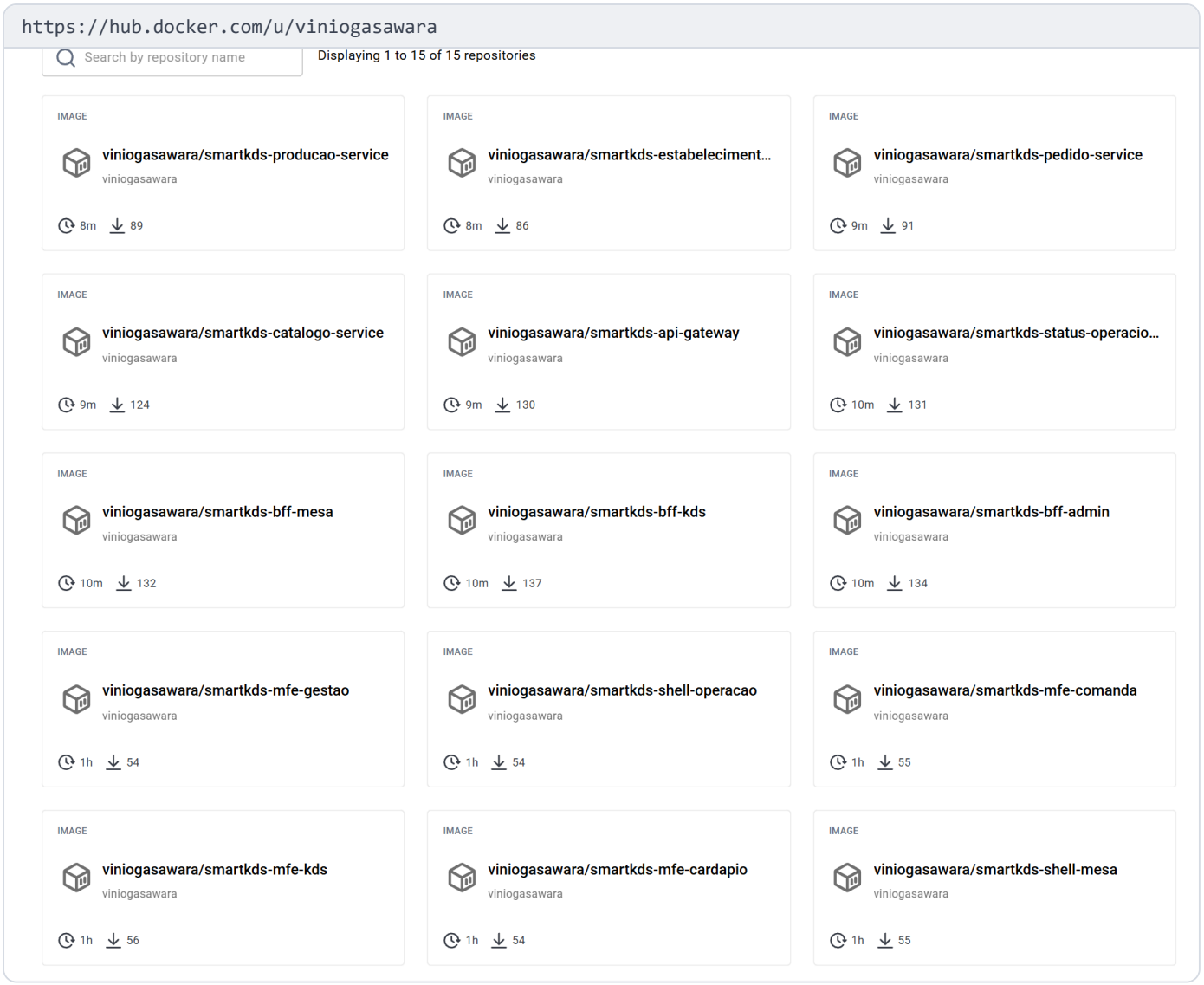
```
$ az containerapp list -g rg-smartkds --query "[].{app:name, externo:properties.configuration.ingress.external, imagem:properties.template.containers[0].image}" -o table
App                     Externo    Imagem
----------------------  ---------  ------------------------------------------------------
estabelecimento-service  False      docker.io/viniogasawara/smartkds-estabelecimento-service:v2
catalogo-service         False      docker.io/viniogasawara/smartkds-catalogo-service:v2
pedido-service           False      docker.io/viniogasawara/smartkds-pedido-service:v2
producao-service         False      docker.io/viniogasawara/smartkds-producao-service:v2
bff-mesa                 False      docker.io/viniogasawara/smartkds-bff-mesa:v3
bff-kds                  False      docker.io/viniogasawara/smartkds-bff-kds:v3
bff-admin                False      docker.io/viniogasawara/smartkds-bff-admin:v3
api-gateway              True       docker.io/viniogasawara/smartkds-api-gateway:v2
mfe-comanda              True       docker.io/viniogasawara/smartkds-mfe-comanda:fe-v1
mfe-kds                  True       docker.io/viniogasawara/smartkds-mfe-kds:fe-v1
mfe-cardapio             True       docker.io/viniogasawara/smartkds-mfe-cardapio:fe-v1
mfe-gestao               True       docker.io/viniogasawara/smartkds-mfe-gestao:fe-v1
shell-mesa               True       docker.io/viniogasawara/smartkds-shell-mesa:fe-v1
shell-operacao           True       docker.io/viniogasawara/smartkds-shell-operacao:fe-v1

$ az containerapp env show -n cae-smartkds -g rg-smartkds --query "{modo:properties.environmentMode, local:location}" -o table
Modo              Local
----------------  -------------
WorkloadProfiles  Chile Central

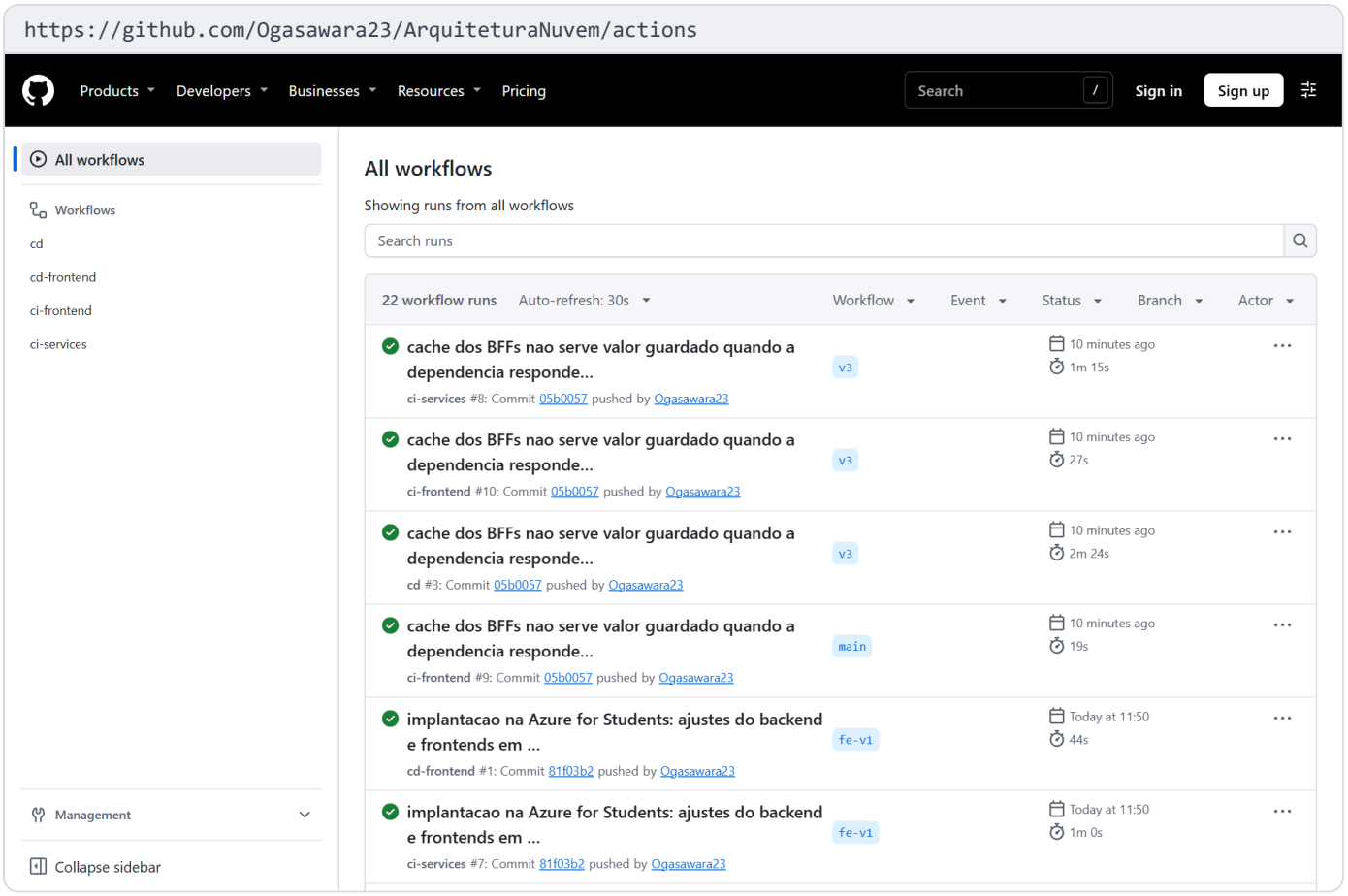
$ az functionapp list -g rg-smartkds --query "[].{function:name, local:location, estado:state}" -o table
Function              Local           Estado
--------------------  --------------  --------
smartkds-status-5560  Canada Central  Running

$ az sql db list -g rg-smartkds -s sql-smartkds-8759 --query "[?name!='master'].{banco:name, sku:currentServiceObjectiveName, estado:status}" -o table
Banco            Sku    Estado
---------------  -----  --------
smartkds_pedido  Basic  Online
```

Print 19 — Azure CLI no grupo rg-smartkds: 14 Container Apps (só gateway e microfrontends com ingresso externo), ambiente em modo WorkloadProfiles, Function em execução e o banco `smartkds_pedido` no Azure SQL



Print 20 — Docker Hub: as 15 imagens `smartkds-*` publicadas pelo GitHub Actions



Print 21 — GitHub Actions: execuções verdes de CI (serviços e frontend) e CD (imagens no Docker Hub)